

Manual de Implantação

None

None

None

Table of contents

1. Notas de Release	3
1.1 Notas de Release	3
2. Manual de Infraestrutura e-SUS AF	6
2.1 Introdução	6
2.2 Público-alvo	6
2.3 Temas abordados	6
3. Componentes	7
3.1 Introdução	7
3.2 COMPONENTES ACESSÓRIOS AO e-SUS AF	9
4. Infraestrutura Recomendada	10
4.1 Infraestrutura de Referência	10
5. Arquitetura	11
5.1 Arquitetura do sistema	11
6. Preparação do Ambiente	12
7. Instalação da aplicação	13
7.1 Instalação com Docker Compose	13
7.2 Instalação com Kubernetes	14
7.3 Load Balancer e Redundância	15
7.4 Banco de Dados	16
7.5 Backup Redict	17
7.6 Checklist Pós-Instalação	18
8. Boas Práticas	19
8.1 Boas Práticas	19
9. Resolvendo Problemas	21
9.1 Troubleshooting	21
10. Referências	22
10.1 Referências	22

1. Notas de Release

1.1 Notas de Release

Este documento contém o histórico de lançamentos do e-SUS AF, detalhando novas funcionalidades, correções de bugs e instruções necessárias para atualização e migração.

1.1.1 [v1.1.1] - 2026-03-02

Novidades

- **Informação de Óbito do Usuário SUS:** Adicionada funcionalidade para registrar informações de óbito do usuário SUS.
- **Cadastro e Gestão de Profissionais de Saúde (Prescritores):** Adiciona recurso para gerenciar informações dos profissionais prescritores no sistema.
- **Exportação em Excel do Relatório de Histórico de Dispensação do Usuário SUS:** Permite exportar dados do histórico de dispensação em formato Excel para facilitar análises e relatórios.

Correções

- Correção de falhas que impactavam fluxos críticos como óbito, dispensação e gestão de perfis.
- Melhorias nos relatórios gerenciais, com adequações de layout e geração em formato Excel.

Mudanças Críticas (Breaking Changes)

- Adição da variável de ambiente `MAIL_SECURE` para habilitar ou desabilitar o TLS para e-mail (implementação de compatibilidade com STARTTLS).
 - Adição de novas entidades de banco de dados.
-

1.1.2 Instruções de Atualização

Para atualizar para esta versão, siga os passos abaixo de acordo com sua infraestrutura.

1. Atualização de Imagens (Registry)

As novas imagens estão disponíveis no Gitlab Container Registry. Atualize seu arquivo de implantação com as tags específicas da versão.

Docker Compose: No arquivo `docker-compose.yml`, altere as tags das imagens:

```
services: web: image: gitlab.inova-af.dev.br:5050/inova-af/esus-af/e-susaf-web:v1.1.1 api: image: gitlab.inova-af.dev.br:5050/inova-af/esus-af/e-susaf-api
```

Kubernetes: Atualize os deployments via kubectl:

```
kubectl set image deployment/esus-web web=gitlab.inova-af.dev.br:5050/inova-af/esus-af/e-susaf-web:v1.1.1 kubectl set image deployment/esus-api api=gitlab
```

2. Migração de Banco de Dados

Esta versão contém migrações de esquema que devem ser aplicadas **antes** de subir a nova versão da aplicação.

Passo 1: Backup de Segurança Sempre realize um backup antes de iniciar a migração:

```
pg_dump -U postgres -h <db_host> -F c -b -v -f backup_antes_v1.0.5.sqlc esus
```

Passo 2: Execução das Migrações As migrações deverão ser executadas manualmente, respeitando a ordem de execução definida no repositório database/postgres.

ATENÇÃO

Os arquivos DDL do e-SUS AF estão no repositório em `/database/postgres` e os arquivos de DML estão no repositório `database/data`.

Os arquivos estão organizados em ordem cronológica. Para o sucesso da migração, siga a ordem cronológica dos arquivos partindo do arquivo subsequente ao último executado.

3. Verificação

Após verificar a última atualização em banco de dados, execute em ordem os arquivos DDL e DML. Por exemplo, se a versão implantada em sua infraestrutura é a `v1.0.5` e todos os arquivos foram executados com sucesso para a versão em questão (deverão ter sido executados os arquivos e anteriores aos citados a seguir: DML - 20251027211540-Ajustes-dominios-bug.sql e DDL - 20250815104637-configuracao_si_bnafar.sql), execute os arquivos DDL e DML listados abaixo na ordem em que estão apresentados.

DDL

- 20251117205241-ajuste_prescritor_conselho.sql
- 20260115174000-add_sigla_tipo_movimentacao.sql

- 20260227133926-ajuste-numero-conselho-crf.sql

DML

- 20251201210012-permissoes-prescritor.sql
- 20260115174001-update_siglas_tipo_movimentacao.sql
- 20260122104835-resources-config-sibnafar.sql
- 20260122163945-Insert-Config-SIBNAFAR.sql
- 20260227174856-Insert_entrada_por_doacao.sql

1.1.3 Plano de Rollback

Caso ocorra algum erro crítico após a atualização: 1. Reverta as imagens para a versão anterior (sugerida a `v1.0.5`). 2. Se houve alteração destrutiva no banco, restaure o backup realizado no Passo 1:

```
pg_restore -U postgres -h <db_host> -d esus -v backup_antes_v1.0.5.sqlc
```

3. Entre em contato com a equipe do Inova-AF para apoio e suporte.

2. Manual de Infraestrutura e-SUS AF



2.1 Introdução

Este documento foi construído para apoiar o processo de implantação do e-SUS AF, com foco na infraestrutura necessária para o funcionamento do sistema. Desta forma, para que seja possível obter o maior aproveitamento deste material, espera-se que o leitor tenha conhecimento básico sobre a arquitetura de sistemas e infraestrutura de TI.

2.2 Público-alvo

Este manual é destinado a desenvolvedores, gestores de configuração e profissionais de TI que atuam na implantação e manutenção do sistema e-SUS AF. O conteúdo é voltado para aqueles que desejam entender melhor a infraestrutura necessária para o funcionamento do sistema, bem como os requisitos técnicos e orientações para instalação e configuração.

2.3 Temas abordados

Serão abordados temas relevantes para a infraestrutura do e-SUS AF, incluindo:

- Descrição da arquitetura do sistema
- Requisitos técnicos
- Componentes da aplicação
- Orientações para instalação e configuração
- Melhores práticas de segurança
- Monitoramento e manutenção

3. Componentes

3.1 Introdução

Este módulo descreve os componentes do sistema **e-SUS AF**, incluindo os componentes acessórios e principais para a aplicação e suas respectivas funções.

Para fins gerais, considere como *Componentes Principais* aqueles que são estritamente necessários para o funcionamento do sistema, tais como interface web, api de serviços, e demais componentes essenciais ao funcionamento da aplicação, e como *Componentes Acessórios*, aqueles que estendem recursos do sistema garantindo funcionalidades adicionais à aplicação.

São considerados componentes obrigatórios:

- Interface WEB
- API de serviços
- Banco de Dados
- Serviço de cache
- Serviço de integração ao SI BNAFAR

Quanto ao conjunto de componentes opcionais, listamos:

- Interface Web de monitoramento
- Serviço de coleta de logs
- Serviço de coleta de métricas
- Interface de dataview
- Serviço de automação, gerenciamento e coordenação de fluxos de dados

3.2 COMPONENTES ACESSÓRIOS AO e-SUS AF

Como relatado no documento de introdução, os componentes acessórios são aqueles que estendem os recursos do sistema e garantem funcionalidades adicionais à aplicação. Estes componentes possibilitam tem funções variadas atuando em inúmeras frentes.

A escolha de quais componentes acessórios utilizar é do gestor de infraestrutura e, para esta definição, deve-se considerar o porte da aplicação, a infraestrutura disponível e as necessidades de monitoramento e gestão do sistema.

A seguir, listamos os componentes acessórios que podem ser utilizados com o e-SUS AF.

3.2.1 Apache Superset

O Apache Superset é uma plataforma de visualização de dados que permite a criação de dashboards e relatórios interativos.

Ele pode ser integrado ao e-SUS AF para fornecer uma interface de visualização dos dados, facilitando a análise e interpretação das informações possibilitando a criação de gráficos, tabelas e outros elementos visuais que ajudam na tomada de decisões.

Faz parte do conjunto de ferramentas de Business Intelligence (BI) e é uma opção popular para visualização de dados em ambientes corporativos.

O banco de dados do e-SUS AF é munido de um conjunto de views que podem ser utilizadas para a criação de dashboards e relatórios no Apache Superset, estas views são criadas e mantidas pelo próprio e-SUS AF, garantindo que os dados estejam sempre atualizados e consistentes.

para além das views disponibilizadas, é possível acessar diretamente as tabelas do banco de dados, permitindo uma flexibilidade maior na criação de visualizações personalizadas sw forma a tirar máximo proveito dos recursos avançados de filtragem, agrupamento e drill-down, permitindo que os usuários explorem os dados de forma interativa.

O Repositório do BI e-SUS AF oferece uma série de dashboards pré-configurados que podem ser utilizados como ponto de partida para a criação de visualizações personalizadas, facilitando a análise de dados e a tomada de decisões informadas.

```
graph TD
  A[Usuário] --> B[Apache Superset]
  B --> E[Dashboards Pré-configurados]
  B --> F[Dashboards Personalizados]
  E --> C[Views do Banco de Dados e-SUS AF]
  F --> C
  C --> D[Tabelas do Banco de Dados e-SUS AF]
```

4. Infraestrutura Recomendada

4.1 Infraestrutura de Referência

4.1.1 Requisitos Mínimos

Serviço	CPU	Memória RAM	Disco	Observação
API	2 vCPUs	4 GB	20 GB	Ideal em VM ou container isolado
WEB	1 vCPU	2 GB	10 GB	Pode ser compartilhado com a API em ambientes pequenos
Banco de Dados	4 vCPUs	8 GB	SSD 100 GB	Volume persistente recomendado
Redis	1 vCPU	1 GB	1 GB	Utilizado para cache

Requisitos de Infraestrutura (Cloud e On-Premises)

Requisitos mínimos (por componente)

Componente	CPU	Memória	Armazenamento	Observação
API	2 vCPUs	4 GB	20 GB SSD	Banco externo recomendado
WEB	2 vCPUs	2 GB	10 GB SSD	Load Balancer recomendado
Banco de Dados	4 vCPUs	8 GB	100 GB SSD	Discos provisionados ou NVMe
Redis	1 vCPU	1 GB	2 GB	Para cache auxiliar

Requisitos recomendados (Produção - Alta disponibilidade)

Cloud	API	WEB	Banco de Dados	Observação
AWS	t3.medium	t3.small	r5.large + EBS gp3	Multi-AZ e snapshots
Azure	B2ms	B1ms	E4s v5	Storage Premium
GCP	e2-standard-2	e2-small	n2-standard-4	Persistent Disks Regional

Para On-Premises:

- Virtualização recomendada: Proxmox, VMware, KVM
- Load Balancer: HAProxy ou NGINX
- Storage: RAID 10 com discos SSD ou NVMe
- Backup: sistema de snapshots + offsite

5. Arquitetura

5.1 Arquitetura do sistema

A arquitetura do sistema é composta de 2 partes para o sistema, que consiste na camada de apresentação denominada de Serviço Web e o Serviço de API. O sistema é composto dos seguintes componentes:

- *Load Balance* - Componente responsável pela distribuição da carga entre máquinas
- *Serviço Web* - Componente responsável pela disponibilização da camada de apresentação da aplicação
- *Serviço de API* - Componente responsável pela disponibilização dos dados do sistema.
- *Redis* - Componente responsável pelo armazenamento dos tokens de autenticação e cache distribuído.
- *Banco de dados* - Componente de persistência de dados. Nessa versão inicial a compatibilidade testada é com o Banco PostgreSQL.

```
+-----+ +-----+ | Usuário Final |<----->| Load Balancer | +-----+ +-----+ | +-----+
```

6. Preparação do Ambiente

6.0.1 Preparação do Ambiente

Pré-requisitos

Como pré requisito para a instalação do sistema pedimos:

- Orquestrador de container
 - Docker e Docker Compose (em ambiente local ou servidor)
 - Kubernetes >= 1.28 (em ambiente clusterizado)
- Variáveis de ambiente definidas:

```
POSTGRES_USER=postgres POSTGRES_PASSWORD=12345678 REDIS_PASSWORD=123456 REDIS_PORT=6379
```

7. Instalação da aplicação

7.1 Instalação com Docker Compose

Crie o arquivo `docker-compose.yml`:

```
version: '3.5' services: web: container_name: web image: ghcr.io/projeto-af/e-susaf/e-susaf-web:latest ports: - "8000:80" environment: VAR__API_URL: "http
```

Execução

Para executar, siga com o comando:

```
docker-compose up -d
```

7.2 Instalação com Kubernetes

7.2.1 Deployment WEB

`web-deployment.yaml`

```
apiVersion: apps/v1 kind: Deployment metadata: name: web spec: replicas: 2 selector: matchLabels: app: web template: metadata: labels: app: web spec: cont
```

7.2.2 Deployment API

`api-deployment.yaml`

```
apiVersion: apps/v1 kind: Deployment metadata: name: api spec: replicas: 3 selector: matchLabels: app: api template: metadata: labels: app: api spec: cont
```

7.2.3 StatefulSet Banco de Dados

Para o banco de dados, a versão preliminar seria um PostgreSQL. Recomendamos um serviço gerenciado de banco de dados. Se necessário por economia ou para disponibilizar um ambiente menor para treinamento ou homologação do sistema, pode-se instalar um banco de dados diretamente no cluster. Segue um exemplo do yaml para isso.

`db-statefulset.yaml`

```
apiVersion: apps/v1 kind: StatefulSet metadata: name: db spec: serviceName: "db" replicas: 1 selector: matchLabels: app: db template: metadata: labels: ap
```

7.3 Load Balancer e Redundância

- O serviço `web` deve expor `type: LoadBalancer` ou ser gerenciado via `Ingress Controller`.
- Replique a `API` e o `WEB` para obter alta disponibilidade.
- Utilize Redis e PostgreSQL com estratégias de failover (Redis Sentinel e Patroni são recomendados).

7.4 Banco de Dados

- Sempre utilizar armazenamento persistente.
- Implementar **backup automatizado** com `pg_dump`, `wal-g` ou serviço gerenciado.
- Se possível, utilizar PostgreSQL como serviço gerenciado (AWS RDS, GCP CloudSQL, etc).

7.4.1 Backup PostgreSQL

```
pg_dump -U postgres -h <db_host> -F c -b -v -f /backups/backup-$(date +%F).sqlc esus
```

Restore PostgreSQL

```
pg_restore -U postgres -h <db_host> -d esus -v /backups/backup-2025-03-27.sqlc
```


7.5 Backup Redis

O Redis salva automaticamente no arquivo `dump.rdb` configurado em `redis.conf`.

Para garantir um backup diário, armazene o arquivo rdb:

```
BGSAVE cp /data/dump.rdb /backup/redis-dump-$(date +%F).rdb
```

7.6 Checklist Pós-Instalação

- Backup automático validado
- Load Balancer com health-check funcional
- Monitoramento ativo e gerando alertas
- TLS implementado
- Secrets e senhas protegidos
- Políticas de rede (firewall ou Kubernetes NetworkPolicy) aplicadas

8. Boas Práticas

8.1 Boas Práticas

- Agende backups diários com `cron`.
- Armazene em local externo (cloud ou storage dedicado).
- Mantenha ao menos 7 dias de retenção e uma cópia off-site.

8.1.1 Segurança e Boas Práticas

- Nunca armazene segredos no `docker-compose.yml` ou manifests do Kubernetes. Utilize:
 - Docker Secrets
 - Kubernetes Secrets + Vault (Hashicorp) ou SOPS
- Acesso SSH restrito e auditado
- Backup criptografado
- Use TLS para todas as comunicações (Ingress, API, DB)
- Configurar políticas de rede (NetworkPolicy no Kubernetes ou firewalls em Docker)
- Configurar autenticação e RBAC nos clusters

8.1.2 Monitoramento e Logging

Para garantir a estabilidade, performance e segurança do sistema, o monitoramento contínuo e o logging centralizado são indispensáveis.

Requisitos de Monitoramento

- **Métricas essenciais:**
 - CPU, memória, disco e rede de todos os serviços
 - Latência e disponibilidade da API
 - Status do banco de dados (replicação, consumo de espaço, performance de queries)
 - Load Balancer (tempo de resposta, erros 4xx/5xx)

Stack Recomendada

- **Prometheus + Grafana:** coleta e visualização de métricas.
- **Loki ou ELK Stack (Elasticsearch, Logstash, Kibana):** logging centralizado.
- **Alertmanager:** envio de alertas por e-mail, Slack ou outros canais.

Boas Práticas

- Retenção de logs adequada (mínimo 30 dias em produção).
- Logs armazenados fora da máquina principal (volumes ou serviços externos).
- Alertas de thresholds críticos bem definidos.

Testes de funcionalidade

- Consumo de API via Postman ou Insomnia
- Fluxo de autenticação e funcionalidades principais

Testes de carga (opcional)

- `k6`, `wrk`, `locust` para simular usuários simultâneos

Verificação de monitoramento e alertas

- Simular falha de serviço para validar recebimento de alertas
- Verifique status:

9. Resolvendo Problemas

9.1 Troubleshooting

Problema	Causa Comum	Solução
API não conecta no Banco	Variáveis de ambiente incorretas	Verifique a definição de env vars
Porta não acessível	Firewall bloqueando	Libere as portas 8000 e 3001
Load Balancer não sobe	Cluster mal configurado	Verifique logs e provisionamento do cloud provider

9.1.1 Comandos úteis

Função	Comando
Verificar logs (Docker)	<code>docker-compose logs -f</code>
Verificar containers	<code>docker ps</code>
Verificar pods (K8s)	<code>kubectl get pods</code>
Logs de um pod	<code>kubectl logs <pod></code>
Acesso ao banco	<code>psql -h <db_host> -U <user> -d <database></code>

9.1.2 Diagnóstico de problemas comuns

- Variáveis de ambiente ausentes ou mal configuradas
- Consumo elevado de recursos
- Falha de conectividade entre serviços
- Volume não persistente no banco de dados

10. Referências

10.1 Referências

- Docker: <https://docs.docker.com>
- Kubernetes: <https://kubernetes.io/docs>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Prometheus: <https://prometheus.io>
- Grafana: <https://grafana.com>
- Elastic Stack (ELK): <https://www.elastic.co/elastic-stack>
- HAProxy: <https://www.haproxy.org/>
- Segurança em Containers (OWASP): <https://owasp.org/www-project-container-security/>